

FALCON

Fast Adaptive Localization via Correlation & Kalman-filter Observation Network

Signal 13

Abu Bakar Siddique(2305059)

Wahidul Hoque(2305054)

September 26, 2026

Why the name FALCON?

- A falcon locks onto fast-moving prey, tracks it in flight, and re-acquires it if it briefly disappears behind an obstacle.
- That is exactly the behaviour we engineered: **fast local tracking** while the motion is predictable, and **automatic re-acquisition** when it is not.

Why the name FALCON?

- A falcon locks onto fast-moving prey, tracks it in flight, and re-acquires it if it briefly disappears behind an obstacle.
- That is exactly the behaviour we engineered: **fast local tracking** while the motion is predictable, and **automatic re-acquisition** when it is not.
- So we named the project after the bird — and built the acronym around what it actually does.

F

Fast

Search only a small local region, not the whole frame

FALCON — Letter by Letter

F **Fast** Search only a small local region, not the whole frame

A **Adaptive** Working template updates slowly to small appearance change

FALCON — Letter by Letter

F **Fast** Search only a small local region, not the whole frame

A **Adaptive** Working template updates slowly to small appearance change

L **Localization** Finding *where* the target is, frame to frame

FALCON — Letter by Letter

F **Fast** Search only a small local region, not the whole frame

A **Adaptive** Working template updates slowly to small appearance change

L **Localization** Finding *where* the target is, frame to frame

C **Correlation** Custom FFT phase correlation finds the displacement

FALCON — Letter by Letter

F **Fast** Search only a small local region, not the whole frame

A **Adaptive** Working template updates slowly to small appearance change

L **Localization** Finding *where* the target is, frame to frame

C **Correlation** Custom FFT phase correlation finds the displacement

O **Observation** PSR + NCC “sanity checks” on every candidate match

FALCON — Letter by Letter

F **Fast** Search only a small local region, not the whole frame

A **Adaptive** Working template updates slowly to small appearance change

L **Localization** Finding *where* the target is, frame to frame

C **Correlation** Custom FFT phase correlation finds the displacement

O **Observation** PSR + NCC “sanity checks” on every candidate match

N **Network** Full pipeline: Django app, DSP modules, analytics

FALCON — Letter by Letter

F **Fast** Search only a small local region, not the whole frame

A **Adaptive** Working template updates slowly to small appearance change

L **Localization** Finding *where* the target is, frame to frame

C **Correlation** Custom FFT phase correlation finds the displacement

O **Observation** PSR + NCC “sanity checks” on every candidate match

N **Network** Full pipeline: Django app, DSP modules, analytics

Fast Adaptive Localization via Correlation & Kalman-filter Observation Network

FALCON in One Paragraph

A local, browser-based object tracker. The user selects a target (from the video, or from a separate reference image). FALCON tracks it using custom FFT phase correlation, checks every match with PSR and NCC, and uses a Kalman filter to predict motion. If the target is occluded or jumps away, the system switches to a time-sliced global search until it is found again.

Motivation and Problem Addressed

- Many simple trackers work only while the object stays near the previous position.
- In our test cases, the object can be occluded, jump to another place, or appear later from a reference image.
- Searching the full frame every time is possible, but it is too slow for a CPU-based custom FFT pipeline.
- **So the project problem is:** track locally when possible, but re-acquire globally when local tracking fails.

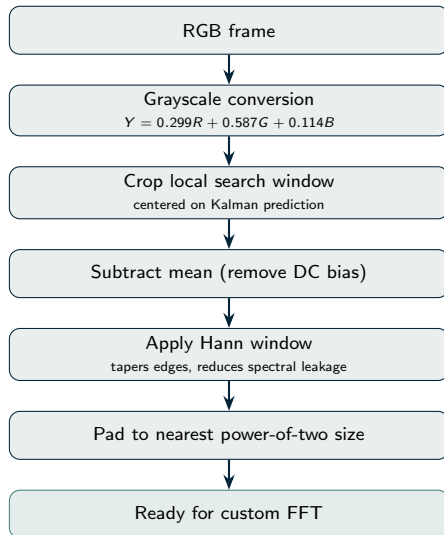
The Core Problems

- ❶ **Tracking efficiency:** Real-time tracking requires CPU-efficient processing. Searching the entire frame for a small target every frame is wasteful.
- ❷ **Noisy correspondence:** Phase correlation always returns a peak, even on background or noise. How do we know when to trust it?
- ❸ **Handling disappearance:** Targets get occluded, jump away, or may not appear until later in the video. A tracker must survive these gaps.
- ❹ **Stability under drift:** Appearance changes accumulate over time. The tracker must not drift away from the true target.

Why Preprocess at All?

- Raw video frames are not ready for FFT-based matching.
- Three problems must be fixed before correlation:
 - ① Color is irrelevant and expensive → convert to grayscale.
 - ② FFT assumes a *periodic* signal → hard crop edges create fake frequencies (*spectral leakage*).
 - ③ FFT works best on power-of-two sized arrays → sizes must be padded/rounded.

Preprocessing Steps



Why Subtract the Mean? (Removing DC Offset)

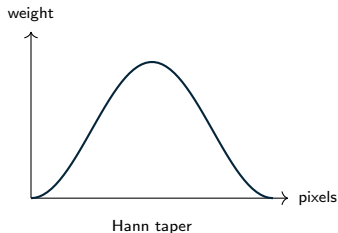
- Before windowing, we subtract the mean pixel value of the crop:

$$I'(x, y) = I(x, y) - \bar{I}, \quad \bar{I} = \frac{1}{N} \sum_{x, y} I(x, y)$$

- $\bar{I}$ (the average brightness) is the signal's **DC offset** — it shows up as a huge spike at frequency (0, 0) in the FFT.
- That spike carries *no* displacement information, but it can dominate the spectrum and overshadow the real correlation peak.
- Removing it first means the FFT only encodes the patch's *structure* (edges, texture) — exactly what phase correlation needs to find the shift.

The Hann Window — Formula

- FFT secretly assumes the crop repeats forever, like tiles.
- A hard rectangular crop creates a sharp jump at the edges → this leaks fake energy into the spectrum.
- The Hann window gently fades the crop to zero at its borders, so the “fake edges” disappear.



$$w(n) = 0.5 - 0.5 \cos\left(\frac{2\pi n}{N-1}\right), \quad n = 0, 1, \dots, N-1$$

For a 2D image patch the window is **separable**:

$$w(x, y) = w(x) \cdot w(y)$$

applied as $I_{\text{windowed}}(x, y) = I'(x, y) \cdot w(x, y)$ right before the FFT.

How far has this pattern moved between two frames?

- Phase correlation answers this directly in the frequency domain.
- A shift in the image becomes a *phase difference* in the Fourier transform — and that phase difference converts cleanly back into a displacement.
- We never search pixel-by-pixel; the FFT gives us the best shift in one shot.

Theory: The Cross-Power Spectrum

- Let F be the FFT of the search patch and T the FFT of the template. The **cross-power spectrum** is:

$$R(u, v) = \frac{F(u, v) T^*(u, v)}{|F(u, v) T^*(u, v)|}$$

- T^* is the complex conjugate of the template's FFT.
- Dividing by the magnitude keeps only the *phase* — this is what makes it a *phase* correlation.
- The correlation surface is the inverse FFT of R :

$$r(x, y) = \mathcal{F}^{-1}\{ R(u, v) \}$$

- The location of the peak of $r(x, y)$ **is** the estimated displacement $(\Delta x, \Delta y)$.

Sub-Pixel Refinement

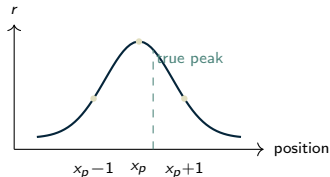
- The correlation surface $r(x, y)$ only lives on **integer** pixel positions — but real motion rarely lands exactly on a whole pixel.
- After finding the strongest integer peak (x_p, y_p) , we fit a small **parabola** through it and its immediate neighbors, separately along each axis.

$$a = r(x_p - 1, y_p), \quad b = r(x_p, y_p), \quad c = r(x_p + 1, y_p)$$

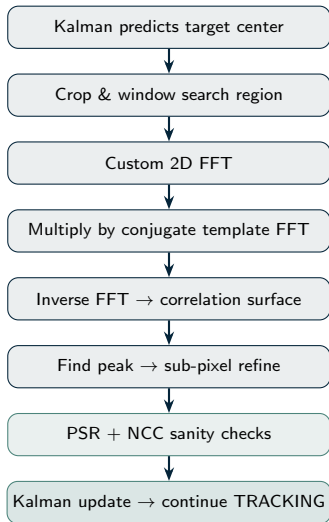
$$\delta_x = \frac{a - c}{2(a - 2b + c)}$$

(same formula along y , using the neighbors above/below the peak instead)

- Final displacement: $(x_p + \delta_x, y_p + \delta_y)$, with $\delta_x, \delta_y \in (-0.5, 0.5)$.



Local Tracking — Frame-by-Frame Flow



Why Each Piece Is Needed

Component	Solves
Phase correlation	"How far has the pattern moved?"
PSR	"Is this peak actually convincing, or just noise?"
NCC	"Does the matched patch really look like the target?"
Kalman filter	"Where should I even look next?"
Global grid search	"What if it's nowhere near the prediction?"

No single piece is sufficient alone — the combination is the contribution.

Our Approach: Constant Velocity Model

- We assume **constant velocity** motion — no acceleration model needed.
- Two phases, each frame:
 - ① **Predict:** “Where should the target be, based on its speed?” → centers the local search window.
 - ② **Update:** If phase correlation + PSR/NCC pass, correct the velocity estimate using that measurement.
- If local tracking fails, velocity is **reset** when we re-acquire the target globally — we don't try to guess teleports or occlusions.

Honest scope: Kalman models *ordinary continuous motion*. It is not designed to predict occlusion or large jumps.

Why We Don't Trust the FFT Peak Blindly

- Phase correlation always returns *some* peak — even on pure background or noise.
- We need two independent “sanity checks” before we trust a match:
 - Is the peak actually **distinctive**? → **PSR**
 - Does the matched region **look like** our target? → **NCC**

PSR — Peak-to-Sidelobe Ratio

- Compares the main peak height against the mean and spread of the *rest* of the correlation surface.
- A tall, isolated peak $\rightarrow$ high PSR $\rightarrow$ confident match.
- A flat, noisy surface $\rightarrow$ low PSR $\rightarrow$ reject it.
- Local threshold used: **PSR** $>$ **10**.

sharp, isolated peak

$$\text{PSR} = \frac{g_{\max} - \mu_{sl}}{\sigma_{sl}}$$

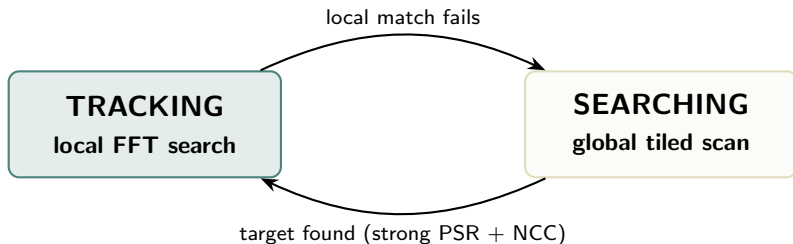
NCC — Appearance Check

- PSR tells us whether the correlation peak is sharp.
- NCC tells us whether the matched patch actually looks like the stored target template.
- This prevents the tracker from trusting a high peak created by a structured background.
- Local threshold used: **$NCC > 0.35$** .

Simple meaning

NCC is a normalized similarity score from -1 to 1 ; closer to 1 means stronger appearance match.

Two-State Design



- Only **two** meaningful states — deliberately.
- “Occluded” is not a separate state: it is simply what *causes* a local match failure and the move into SEARCHING.

Continuous Searching Instead of Final Failure

- The current design keeps the tracker in SEARCHING until the object is re-acquired.
- Reason: with a separate reference image, the selected object may appear much later in the video.
- So “not found yet” is treated as a normal searching condition, not a final failure.
- The system keeps scanning global tiles until a strong PSR/NCC match is found.

The Solution: Time-Sliced Global Search

- Scanning the *whole* frame every video frame is too expensive with our custom (non-library) FFT.
- Instead, while SEARCHING:
 - the frame is divided into **overlapping** grid tiles,
 - only a **few tiles per video frame** are checked (default: 4),
 - this trades a little re-acquisition delay for stable, predictable CPU usage.
- Every tile is checked against the **fixed original reference template** (not the drifted working template) — so re-acquisition always anchors back to the true target.
- As soon as a tile passes strong PSR/NCC thresholds (~ 15 / ~ 0.50), the Kalman filter is **reset** at that location and the system returns to TRACKING.

Limitations — Stated Honestly

- **Translation-focused:** phase correlation is primarily built for translation, not scale/rotation.
- **Not scale-invariant:** a much bigger/smaller target than the template weakens matching.
- **Not rotation-invariant:** large rotation can break PSR/NCC.
- **Single target only:** tracks one selected identity at a time.

Limitations — Continued

- **No semantic understanding:** it matches pixels, not the concept of “a car” or “a ball”.
- **Similar distractors:** a near-identical object nearby can pass PSR/NCC and cause a wrong lock.
- **Global search has latency:** a target visible only very briefly might vanish before its tile is reached.
- **CPU-bound:** the custom FFT is transparent and educational, but slower than optimized library/GPU FFTs.

- User can select a target from the video or provide a separate reference image.
- The app tracks the object locally using custom FFT phase correlation.
- It validates matches using PSR and NCC before updating the tracker.
- If the target is occluded, teleported, or appears later, global tiled search can re-acquire it.
- The Django interface shows the tracking result and the processing pipeline live.

The Core Engineering Idea

Use a **cheap local tracker** while the motion assumption holds,
switch to a **bounded global search** when it doesn't.

- Custom FFT + Phase Correlation
- PSR + NCC appearance validation
- Kalman motion prediction
- Time-sliced global grid search for re-acquisition
- Fixed reference template as an anchor
- Full DSP pipeline visualized live in a Django app

Thank You

Questions?